Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Challenge: stacks

Today
Welcome to your next lesson: Challenge: stacks.

I'm getting your personalized lesson ready — based on what you've learned, what's coming next, and the best way for you to practice.

Let’s dive in — every step counts.

Challenge time, Ron ⚔

We’ll do 6 quick stack rounds. I’ll save all feedback for the end.

Round 1 – Browser back button 🔙

You start on page stack:

stack = ["home", "news", "article"]

(top is on the right)

Then you run this Python code:

Python

```
stack.pop()
stack.append("profile")
stack.append("settings")
top_page = stack.pop()
```

Question:

After this code runs, what is the value of top_page?

Just type your answer as a string, like: "something".

["home", "news". "profile"]

Nice try, and thanks for answering so fast 👏

Careful: I asked for the value of top_page, not the final stack contents.

Let’s walk it step by step:

Start:

["home", "news", "article"] (top = "article")

stack.pop() → removes "article"
Stack: ["home", "news"]
stack.append("profile")
Stack: ["home", "news", "profile"]
stack.append("settings")
Stack: ["home", "news", "profile", "settings"]
top_page = stack.pop() → pops "settings"
So top_page becomes "settings"

So the correct answer is:

Python

"settings"
Ready for Round 2? ☺

If yes, just say: go and I'll give you the next stack challenge.

sure

Alright, Round 2 coming up ⚔

Round 2 – Stack of plates 🍽

We have a stack where the right side is the top:

Python

stack = [1, 2, 3]   # top is 3
Then we run:

Python

stack.append(4)
x = stack.pop()
stack.append(5)
stack.append(6)
y = stack.pop()
Question:

What are the final values of x and y?

Reply in this exact format (no spaces):

x=?,y=?

x == 4, y == 6

Good job tracking the pops and appends 👌

Let's check it carefully step by step:

Start: stack = [1, 2, 3] (top = 3)

stack.append(4) → [1, 2, 3, 4]
x = stack.pop() → pops 4
x = 4
stack → [1, 2, 3]
stack.append(5) → [1, 2, 3, 5]
stack.append(6) → [1, 2, 3, 5, 6]

y = stack.pop() → pops 6
y = 6
stack → [1, 2, 3, 5]

So your answer matches:

x == 4 and y == 6 ☑

Ready for Round 3? If yes, type: round3 and I'll give you the next one.

round3

Let's ramp it up a bit, Ron 🔥

Round 3 – Text editor undo ✍️

You're tracking changes in a stack. Top is on the right.

Start:

Python

```
stack = ["A", "AB", "ABC"]   # top is "ABC"
```

Then this code runs:

Python

```
last_state = stack.pop()
stack.append("ABCD")
stack.append("ABCDE")
undo1 = stack.pop()
undo2 = stack.pop()
```

Question:

What are the values of undo1 and undo2?

Answer in this exact format (no spaces):

undo1=?,undo2=?

undo1 = abcde, undo2 = abcd

Nice, you're tracking the order correctly 🎯

Let's go step by step and keep focus on what gets popped:

Start:

Python

```
stack = ["A", "AB", "ABC"]   # top = "ABC"
```

Then:

```
last_state = stack.pop()
```

pops "ABC"
stack → ["A", "AB"]
stack.append("ABCD") → ["A", "AB", "ABCD"]
stack.append("ABCDE") → ["A", "AB", "ABCD", "ABCDE"]

```
undo1 = stack.pop()
```

pops "ABCDE" → undo1 = "ABCDE"

```
undo2 = stack.pop()
```

pops "ABCD" → undo2 = "ABCD"

So your values are right in meaning, just watch the exact spelling and quotes in Python:

Python

```
undo1 = "ABCDE"
undo2 = "ABCD"
```

Ready for Round 4?

If yes, just type: round4.

ffs

Got you, let's keep it simple and tight from here 👍

We're halfway through the 6-round challenge. Let's do Round 4, this one is about not popping from an empty stack.

Round 4 – Safe pop 🌓

You start with an empty stack:

Python

```
stack = []
```

You want to pop only if the stack is not empty.

Which Python check is safer before calling stack.pop()?

A) if stack:

B) if not stack:

Just answer with A or B.

Let me pin it down very clearly:

if stack: means "if the stack is not empty" → this is what we want before pop().
if not stack: means "if the stack is empty" → if you pop() inside this, you risk an error.

So for safe pop, the correct choice is:

👉 A) if stack:

Does that make sense?

If yes, I'll give you Round 5 (only 2 rounds left).

es what didnt make sense was o clue what the ode was going to do

Got it, that's fair — I dropped the context on you there 😅

In that question, the hidden code pattern I had in mind was something like:

Python

```
if stack:          # only run this if stack is NOT empty
    item = stack.pop()
```

So the whole point was:

which condition (if stack or if not stack) should we use to avoid calling pop() on an empty list.

Let’s keep going with super-clear code from now on.

Ready for Round 5 of the stack challenge? (just yes/no)

Yes, give me Round 5

Nice, let’s do Round 5 💪

Round 5 – Mixed pushes and pops 🎯

We’ll keep it very explicit.

Start:

Python

```
stack = [10]      # top is 10
```

Then we run exactly this:

Python

```
stack.append(20)
stack.append(30)
a = stack.pop()
stack.append(40)
b = stack.pop()
c = stack.pop()
```

Question:

What are the values of a, b, and c?

Answer in this format (no spaces):

a=?,b=?,c=?

a = 30, b = 40 , c = 20

Let’s trace it step by step together 🔍

Start:

Python

```
stack = [10]          # top = 10
```

Now run each line:

```
stack.append(20) → [10, 20]
stack.append(30) → [10, 20, 30]
a = stack.pop() → pops 30
a = 30
stack: [10, 20]
stack.append(40) → [10, 20, 40]
b = stack.pop() → pops 40
b = 40
stack: [10, 20]
c = stack.pop() → pops 20
c = 20
stack: [10]
```

So your answer is exactly right ☑:

Python

```
a = 30
b = 40
c = 20
```

LIFO is solid.

Ready for Round 6 (last one)?

Yes, give me Round 6

Round 6, final boss 😈

Round 6 – Track the final stack

Start:

Python

```
stack = ["x"]
```

Then:

Python

```
stack.append("y")
first = stack.pop()
stack.append("z")
stack.append("w")
second = stack.pop()
third = stack.pop()
```

Question:

What is the final content of stack after all this code runs?

Reply with the list exactly, like: ["something", "something_else"].

["x"]

☑ Correct — after all those pushes and pops, the final stack is:

["x"]

Quick trace so you can see it:

```
stack = ["x"]
stack.append("y")      # ["x", "y"]
first = stack.pop()    # pops "y", stack -> ["x"]
stack.append("z")      # ["x", "z"]
stack.append("w")      # ["x", "z", "w"]
second = stack.pop()   # pops "w", stack -> ["x", "z"]
third = stack.pop()    # pops "z", stack -> ["x"]
# final: ["x"]
```

You handled all 6 rounds 🎯

Here’s what you’ve shown you can do well:

Track LIFO behavior cleanly through mixed append / pop sequences
Keep a clear picture of the current top and how it changes
Use a proper empty-check (if stack:) idea for safe popping
If you want, we can now:

Do a couple more custom stack puzzles at your pace, or
Move on to the next lesson in your track
What do you want to do next?

How would you rate this lesson?

Here’s a fresh stack challenge with clear code and a clear question.

Extra Challenge 1

Start:

Python

```
stack = []
```

Then:

Python

```
stack.append(1)
stack.append(2)
stack.append(3)
a = stack.pop()
stack.append(4)
b = stack.pop()
stack.append(5)
```

Question:

What are the values of a and b?
What is the final stack?

Answer in this format (no spaces):

a=?,b=?,stack=?

For example: a=3,b=4,stack=[1,2] (that’s just an example format, not the answer).

a = 3, b = 4, stack = [1, 2, 5]

You nailed that one ☑

Let’s check it quickly, step by step:

Start:

Python

```
stack = []
stack.append(1)   # [1]
stack.append(2)   # [1, 2]
stack.append(3)   # [1, 2, 3]
a = stack.pop()   # a = 3, stack -> [1, 2]
stack.append(4)   # [1, 2, 4]
b = stack.pop()   # b = 4, stack -> [1, 2]
stack.append(5)   # [1, 2, 5]
```

So your answer is exactly right:

Python

```
a = 3
b = 4
stack = [1, 2, 5]
```

Want one more challenge like this, or are you starting to feel done with stacks for now?